



- ▶ Ubuntu por medio de WSL
- ▶ Comandos básicos

Bibliografía

- *The Linux command line. A complete introduction*, William Shotts.

WSL

La programación científica se hace más productiva y agradable con un entorno más adecuado. En esta dirección, el sistema operativo Linux ofrece un conjunto de herramientas y utilidades que facilitan la programación, la simulación y el cálculo científico. Además, permiten tener control total sobre el sistema operativo y la posibilidad de personalizar el entorno de desarrollo según las necesidades que tengamos. Sin mencionar que, con solo un par de comandos, podemos instalar lo que necesitemos para programar, sin necesidad de tener que buscar archivos sospechosos .exe en internet.

En la clase de hoy vamos a habilitar el entorno Linux en Windows utilizando WSL (*Windows Subsystem for Linux*) e instalar una distribución de Linux. Aunque, personalmente recomiendo instalar directamente una distribución de Linux en una partición de del disco duro.

0.0 Habilitar la virtualización Antes de instalar WSL, es necesario asegurarse de que la virtualización esté habilitada en el sistema. Esta opción permite que Windows ejecute entornos como Linux de manera eficiente.

0.1 Verificar si la virtualización está habilitada

- Presiona Ctrl + Shift + Esc para abrir el Administrador de tareas.
- Ir a la pestaña **Rendimiento**.
- Seleccionar **CPU**.
- Busca el campo **Virtualización**.

Si el campo de virtualización aparece como **Habilitado**, podemos continuar con el paso 1. Si aparece como **Deshabilitado**, debemos activarlo desde la BIOS/UEFI.

0.2 Habilitar la virtualización en la BIOS/UEFI

- Reinicia el computador.
- Durante el arranque, presiona la tecla correspondiente para entrar a la BIOS (comúnmente F2, F10, DEL o ESC).
- Busca una opción relacionada con:
 - *Intel Virtualization Technology (VT-x)*
 - *AMD-V*
- Activa esta opción (**Enabled**).
- Guarda los cambios y reinicia el sistema.

0.3 Verificación final Después de reiniciar, vuelve al Administrador de tareas y confirma que la virtualización aparece como **Habilitada**.

1.0 Habilitar WSL

- Presiona la tecla de Windows
- Escribe PowerShell
- Haz clic derecho y selecciona “Ejecutar como administrador”
- Ejecuta el siguiente comando para habilitar WSL y la característica de máquina virtual:

```
1 wsl --install
```

2.0 Instalar la distro

- Listamos todas las distros disponibles con el comando:

```
1 wsl --list --online
```

- Instalamos Ubuntu con el comando:

```
1 wsl --install -d ubuntu
```

- Acá creamos el usuario y asignamos una clave. Ya queda lista la instalación. Ahora reiniciamos el pc y nos debe aparecer el logo de ubuntu. Lo ponemos como acceso directo en el escritorio e ingresamos a él. Inmediatamente se nos abre una consola.

Para verificar la instalación, podemos ejecutar el siguiente comando en PowerShell:

```
1 wsl
```

Si todo está correcto, se abrirá el entorno Linux. También puedes verificar las distribuciones instaladas:

```
1 wsl -l -v
```

Si quieres cerrar WSL completamente, puedes usar:

```
1 wsl --shutdown
```

Comandos básicos en bash

Antes de comenzar a instalar paquetes y herramientas, es recomendable actualizar el sistema para asegurarnos de tener las últimas versiones y correcciones de seguridad.

```
1 sudo apt update
2 sudo apt upgrade -y
```

El comando `sudo` permite ejecutar las órdenes como administrador/a. El comando `update` actualiza la lista de paquetes disponibles y el comando `upgrade` los actualiza. `-y` se usa para decir sí a todo.

Al abrir Ubuntu en WSL, nos ubicamos por defecto en su directorio personal, conocido como *home*. Para verificar en donde estamos parados, se puede ejecutar:

```
1 pwd
```

La salida será similar a:

```
1 /home/usuario
```

El directorio `/home/longas` es el espacio personal de trabajo. Aquí es donde creamos archivos y carpetas y donde se guardan las configuraciones del usuario.

Para ver los archivos y carpetas en el directorio actual:

```
1 ls
```

nos muestra el contenido del directorio,

```
1 Desktop Documents Downloads Dropbox guardar_okular.sh HEPTools Library
2 Music Pictures Public restaurar_okular.sh Software Templates Videos
```

Para ver más detalles, podemos usar:

```
1 ls -la
```

cuya salida es más o menos algo como:

```
1 -rw-----. 1 longas longas 15707 Aug  5 16:17 .bash_history
2 -rw-r--r--. 1 longas longas   18 Jul 22 2025 .bash_logout
3 -rw-r--r--. 1 longas longas  425 Dec 26 2025 .bash_profile
4 -rw-r--r--. 1 longas longas  837 Dec 26 2025 .bashrc
5 drwxr-xr-x. 1 longas longas 1876 Aug  5 11:26 Desktop
6 drwxr-xr-x. 1 longas longas   78 Jul 15 07:45 Documents
7 drwxr-xr-x. 1 longas longas 3616 Aug  5 11:26 Downloads
8 drwx-----. 1 longas longas  366 Aug  5 11:11 .dropbox
9 drwx-----. 1 longas longas 3694 Aug  5 11:11 Dropbox
10 drwxrwxr-x. 1 longas longas   88 Aug  4 05:47 .dropbox-dist
11 -rw-r--r--. 1 longas longas  561 Jan 20 2026 .emacs
12 drwx-----. 1 longas longas   28 May 19 15:38 .emacs.d
13 -rw-r--r--. 1 longas longas   52 Feb 27 09:02 .gitconfig
14 -rwxr-xr-x. 1 longas longas  229 Jun 14 09:13 guardar_okular.sh
15 drwxr-xr-x. 1 longas longas  756 Apr 16 07:06 HEPTools
16 drwxr-xr-x. 1 longas longas   30 Dec 25 2025 .ipython
17 drwxr-xr-x. 1 longas longas   38 Dec 28 2025 .juliaup
18 drwxr-xr-x. 1 longas longas   16 Mar 16 07:13 .jupyter
```

Este comando muestra los archivos ocultos, que comienzan con un punto (.) y suelen usarse para configuraciones del sistema o del usuario. En la salida anterior, la primera columna indica los permisos del archivo, la segunda columna indica el número de enlaces, la tercera y cuarta columnas indican el propietario y el grupo del archivo, la quinta columna indica el tamaño del archivo en bytes, la sexta columna indica la fecha y hora de la última modificación, y la séptima columna indica el nombre del archivo o carpeta. El comando `ls` se puede combinar con otras opciones, acá les dejo el resumen,

```
1 ls          # Vista simple
2 ls -a       # Incluye archivos ocultos
3 ls -l       # Detalle completo (permisos, tamaño, fecha)
4 ls -la      # Todo + ocultos + detalle
5 ls -lt      # Ordenado por fecha (recientes arriba)
6 ls -ltr     # Ordenado por fecha inverso (recientes abajo)
7 ls -lh      # Tamaño en MB/GB inteligibles
```

Cabe anotar que también es posible acceder a los archivos de Windows. Por ejemplo, el disco C: se encuentra en:

```
1 /mnt/c
```

Por lo tanto, la carpeta:

```
1 C:\Users\usuario
```

corresponde en Linux a:

```
1 /mnt/c/Users/usuario
```

Cuando abrimos el power shell de windows e iniciamos `wsl`, la instrucción no nos lleva a nuestro home de linux, sino al home de windows. Si queremos pasar a nuestro "verdadero" home, debemos dar el comando `cd`. Se recomienda trabajar dentro del directorio `/home/usuario` para evitar problemas de rendimiento y permisos.

Continuando con los comandos básicos, podemos crear un directorio con:

```
1 mkdir nombre_directorio
```

Para cambiar de directorio:

```
1 cd nombre_directorio
```

Para regresar al directorio anterior:

```
1 cd ..
```

Para eliminar un archivo:

```
1 rm nombre_archivo
```

Para eliminar un directorio con contenido:

```
1 rm -r nombre_directorio
```

Para copiar un archivo:

```
1 cp archivo_origen archivo_destino
```

Para mover un archivo:

```
1 mv archivo_origen archivo_destino
```

Existen una cantidad de comandos útiles en Linux; de hecho, podemos usar un lenguaje llamado *shell scripting*, con su propia sintaxis. Por ejemplo, abrimos un editor de texto

```
1 emacs mundo.sh
```

y escribimos

```
1 #!/bin/bash
2 echo "hola mundo cruel"
```

guardamos y cerramos el editor.

```
1 ctrl + x + s
2 ctrl + x + c
```

Luego damos permisos de ejecución al archivo:

```
1 chmod u+x mundo.sh
```

y ejecutamos en la terminal:

```
1 ./mundo.sh
```

Sin embargo, en el curso no vamos a profundizar en el tema de *shell scripting*, pues no es el objetivo. Es bueno saber que existe y que es muy útil para automatizar tareas.

Instalación de herramientas básicas Una vez que el sistema está actualizado, el siguiente paso es instalar las herramientas necesarias para programación, compilación y trabajo científico. Si ejecutamos el siguiente comando:

```
1 sudo apt install build-essential git python3 python3-pip gcc gfortran -y
```

Instalaremos un conjunto de herramientas necesarias para la programación científica.

- **build-essential**: herramientas básicas de compilación (make, g++, etc.)
- **git**: sistema de control de versiones
- **python3**: lenguaje de programación Python
- **python3-pip**: gestor de paquetes de Python
- **gcc**: compilador de C
- **gfortran**: compilador de Fortran

Si queremos comprobar que las herramientas están correctamente instaladas:

```
1 gcc --version
2 gfortran --version
3 python3 --version
4 git --version
```

Posteriormente, podemos instalar librerías científicas en Python para trabajo numérico y científico. Sin embargo, lo haremos en lo se conoce como entornos virtuales, que nos permiten tener diferentes versiones de librerías y paquetes para distintos proyectos sin conflictos,

```
1 sudo apt install python3-venv -y
```

Finalmente, procedemos a instalar una distribución completa de $\text{\LaTeX}$:

```
1 sudo apt install texlive-full -y
```

que instala

- Compiladores (pdflatex, xelatex, etc.)
- Paquetes matemáticos (amsmath, amssymb)
- Soporte para figuras, tablas y bibliografía
- Clases de documentos y estilos adicionales

Esto garantiza que prácticamente cualquier documento $\text{\LaTeX}$ pueda compilarse sin errores por falta de paquetes, aunque tiene la desventaja de que ocupa varios GB de espacio en disco y puede tardar varios minutos en instalarse dependiendo de la velocidad de la conexión a internet. Si se desea algo más liviano es suficiente

```
1 sudo apt install texlive-latex-base texlive-latex-recommended
2 texlive-latex-extra texlive-lang-spanish texlive-lang-english
```

Una vez finalizada la instalación, se puede verificar ejecutando:

```
1 pdflatex --version
```

En la actualidad, existen editores en línea como [Overleaf](#), pero, tener $\text{\LaTeX}$ instalado localmente permite trabajar sin conexión y con mayor control del entorno.

Integración de VS Code con WSL

Vamos a usar Visual Code como editor, pero integrado a wsl. Para ello, seguimos los siguientes pasos:

1. **Instalación de la extensión requerida:** Abra Visual Studio Code en Windows, vaya al menú de extensiones (Ctrl + Shift + X) y busque e instale la extensión oficial llamada **WSL** (desarrollada por Microsoft).
2. **Abrir VS Code directamente desde la terminal de WSL:** En la terminal de Linux, navegue hasta la carpeta de trabajo deseada y ejecute el comando `code .` Ejemplo:

```
1 cd ~/Ejercicios
2 code .
3
```

Para confirmar que VS Code está enlazado adecuadamente con WSL Verifique que en la esquina inferior izquierda de la ventana de VS Code aparezca un recuadro de color (verde o azul) con el texto WSL: < distribución> (por ejemplo, WSL: Ubuntu). Abra la terminal dentro de VS Code usando y compruebe que la línea de comandos responda al shell de Linux (Bash) y no a PowerShell ni al Símbolo del sistema de Windows.

Instalación de Extensiones de Python y Jupyter

Para desarrollar código y ejecutar *Notebooks* de manera integrada, hacemos lo siguiente:

1. **Asegurar la conexión a WSL:** Abra VS Code desde la terminal de Linux con `code .` para que la ventana esté conectada al entorno WSL (compruebe el indicador en la esquina inferior izquierda).
2. **Instalar la extensión de Python:**
 - Vaya a la sección de extensiones (`Ctrl + Shift + X`).
 - Busque **Python** (de Microsoft).
 - Haga clic en el botón **"Install in WSL: <distribución>"** (Instalar en WSL). Esto habilitará el coloreado de sintaxis, el autocompletado (*IntelliSense*) y la depuración de scripts `.py`.
3. **Instalar la extensión de Jupyter:**
 - Busque la extensión **Jupyter** (de Microsoft).
 - Asegúrese de instalarla también dentro del entorno WSL.
 - Esto le permitirá crear, editar y correr celdas de *Jupyter Notebooks* (`.ipynb`) directamente dentro de VS Code usando el kernel de Python instalado en Linux.

Cambiar apariencia

```
1 Ctrl + k
2 Ctrl + T
```

Allí buscas el theme que te guste

Preferencias

Los atajos vienen con los usuales. Yo en ocasiones prefiero algunos de emacs. Podemos cambiarlos como queramos

```
1 Presiona Ctrl + Shift + P
2 Escribe Preferences: Open Keyboard Shortcuts (JSON) y dale Enter.
```

Aparecerá un archivo como este

```
1 // Place your key bindings in this file to override the defaults
2 [
3   {
4     "key": "ctrl+x ctrl+s",
5     "command": "workbench.action.files.save"
6   },
7   {
8     "key": "ctrl+w",
9     "command": "editor.action.clipboardCutAction",
10    "when": "editorTextFocus && !editorReadOnly"
11  },
```

```
12 {
13     "key": "ctrl+shift+enter",
14     "command": "editor.action.toggleColumnSelection"
15 },
16 {
17     "key": "ctrl+x ctrl+c",
18     "command": "workbench.action.closeActiveEditor"
19 }
20
21 ]
```

Eso significa que cambié el guardar archivo y el cerrar archivo. Note que el signo menos es para liberar ese atajo.

También podemos cambiar las preferencias de VS Code manualmente, por ejemplo

- 1 Presiona Ctrl + Shift + P
- 2 Escribe Preferences: Open User Settings (JSON) y dale Enter.

Aparecerá un archivo como este

```
1 {
2     "workbench.colorTheme": "Solarized Dark",
3     "python.defaultInterpreterPath": "/usr/bin/python3",
4     "[python]": {
5         "editor.defaultFormatter": "ms-python.black-formatter",
6         "editor.formatOnSave": true
7     },
8     "ltx.language": "auto",
9     "ltx.enabled": [
10     "latex",
11     "markdown",
12     "plaintext"
13 ],
14     "ltx.dictionary": {
15     "es": [],
16     "en-US": [],
17     "pt-BR": []
18 },
19     "editor.tabCompletion": "on",
20     "ltx.completionEnabled": true,
21     "latex-workshop.view.pdf.internal.synctex.keybinding": "double-click",
22
23     "latex-workshop.latex.tools": [
24     {
25         "name": "xelatex",
26         "command": "xelatex",
27         "args": [
28             "-synctex=1",
29             "-interaction=nonstopmode",
30             "-file-line-error",
31             "%DOC%"
32         ]
33     }
34 ]
35 }
```

```
32     ]
33   }
34 ],
35 "latex-workshop.latex.recipes": [
36   {
37     "name": "xelatex view",
38     "tools": [
39       "xelatex"
40     ]
41   }
42 ],
43 "git.autofetch": true
44 }
```

Problemas:

1. Cree el directorio Ejercicios en su directorio personal (*home*).
2. Dentro de Ejercicios, cree los siguientes directorios: Computadores, Personal, Basura, Universidad, Imagenes, Documentos y Tareas.
3. Dentro del directorio Tareas, cree los subdirectorios Tareas_1 y Tarea_2.
4. En la raíz del directorio Ejercicios, cree los siguientes archivos con el contenido indicado:
 - libreta_telefonica.txt: Nombre y teléfono de 3 amigos.
 - materias.txt: Lista de las materias matriculadas durante el semestre.
 - carta: Un mensaje corto a un amigo.
 - cancion.1, cancion.2, cancion.3: Una estrofa de una canción conocida en cada archivo.
 - temario.clase1: Temario general de la primera clase del curso.
 - animales: El nombre de 5 animales.
5. Borre el directorio Tarea_2.
6. Copie los archivos de canciones en el directorio Basura.
7. Borre los archivos de canciones del directorio Ejercicios.
8. Copie todos los archivos con extensión .txt en el directorio Documentos.
9. Mueva el archivo materias.txt al directorio Universidad.
10. Mueva el archivo animales al directorio Basura.
11. Consultar el comando `less` y utilizarlo para revisar el contenido del archivo `temario.clase1`.
12. Mueva todo el directorio Documentos a la carpeta Basura.